

数据库部分详解与答辩准备文档

本文档面向课程答辩使用，重点解释本项目数据库部分的整体设计、每个 SQL 脚本的作用、主要代码逻辑、为什么这么做、可替代方案，以及老师可能追问的问题。

项目数据库名：`campus_activity_db`

项目定位：校园百事通系统，包含活动报名签到、共享物资借还、二手交易、社区互动和统计查询。数据库部分承担了数据存储、业务约束、并发控制、自动化处理、统计聚合等职责。

1. 数据库全局设计思路

1.1 为什么数据库是项目重点

这个项目不是单纯做页面展示，而是一个数据库课程大作业，所以数据库需要体现：

1. 数据模型完整：有用户、角色、社团、学生、活动、报名、签到、物资、借用单、处罚、商品、订单、帖子、评论、点赞等实体。
2. 表关系清晰：大量使用主键、外键、唯一约束，把业务实体连接起来。
3. 数据合法性控制：使用 `CHECK`、`UNIQUE`、外键、触发器限制非法数据。
4. 事务能力：关键业务写操作需要同时修改多张表，不能只成功一半。
5. 触发器：报名时自动判断活动状态、报名截止、容量和候补队列。
6. 存储过程：活动结算、活动删除、候补转正等复杂操作封装到数据库层。
7. 视图：把复杂统计查询封装成可复用查询对象。
8. 事件：自动定时处理逾期借用单。
9. 索引：提高常用筛选和统计查询速度。

1.2 全局数据流

用户在前端页面提交表单，Flask 路由接收请求，通过 PyMySQL 执行 SQL。数据库执行插入、更新、调用存储过程或查询视图，然后 Flask 将结果渲染回页面。

典型流程：

学生点击报名

```
-> Flask 路由 /activity/register/<activity_id>
-> INSERT INTO activity_registration
-> MySQL 触发器 trg_before_insert_registration 自动检查规则
-> 成功则 confirmed 或 waiting
-> 失败则 SIGNAL 抛出错误
-> Flask 捕获异常并显示友好提示
```

典型结算流程：

社团/管理员点击活动结算

```
-> Flask 路由 /activity/finish/<activity_id>
```

```
-> CALL sp_finish_activity(activity_id)
-> 存储过程开启事务
-> 锁定活动行
-> 修改活动状态
-> 给签到学生加分
-> 给缺席学生扣信用并生成处罚
-> 处理逾期借用单
-> COMMIT
```

2. SQL 脚本整体结构

数据库脚本在 `sql/` 目录中：

文件	作用
<code>00_reset_database.sql</code>	删除旧数据库并重建，适合全量初始化
<code>01_create_database.sql</code>	只创建数据库，不删除已有数据
<code>02_create_tables.sql</code>	创建核心业务表、约束和索引
<code>03_insert_init_data.sql</code>	插入演示用户、活动、物资、报名、签到数据
<code>04_trigger.sql</code>	创建触发器：报名控制、审计日志
<code>05_procedure.sql</code>	创建存储过程：候补转正、活动结算、取消活动删除
<code>06_view.sql</code>	创建统计视图
<code>07_event.sql</code>	创建定时事件：自动标记逾期借用单
<code>08_add_trade_community.sql</code>	增加二手交易和社区模块表
<code>09_add_post_like.sql</code>	增加帖子点赞表
<code>10_add_borrow_order_uid.sql</code>	给旧库补借用单 UID
<code>99_bootstrap_all.sql</code>	命令行一键初始化，使用 <code>SOURCE</code>
<code>99_bootstrap_workbench.sql</code>	Workbench 兼容的一键初始化脚本

2.1 为什么拆成多个 SQL 文件

这样做的好处：

1. 模块清晰：建表、初始化数据、触发器、过程、视图分开。
2. 便于调试：某个模块出问题时可以单独执行对应文件。
3. 便于课程展示：老师问触发器或存储过程时可以直接定位到文件。
4. 支持增量升级：`08`、`09`、`10` 是后续扩展脚本，不影响早期核心表。

可替代方案：

1. 所有 SQL 写在一个文件中：简单，但不利于维护和讲解。
2. 使用数据库迁移工具，如 Alembic/Flyway：更专业，但课程项目复杂度会升高。

3. 使用 ORM 自动建表：开发快，但不利于展示原生 SQL、触发器、存储过程。

3. 00_reset_database.sql：重置数据库

核心代码：

```
DROP DATABASE IF EXISTS campus_activity_db;

CREATE DATABASE campus_activity_db
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_0900_ai_ci;

USE campus_activity_db;
```

3.1 每句代码作用

```
DROP DATABASE IF EXISTS campus_activity_db;
```

删除旧数据库。如果数据库不存在，不报错。适合课程演示前重新初始化，保证数据状态干净。

```
CREATE DATABASE campus_activity_db ...
```

重新创建数据库，指定字符集为 `utf8mb4`，排序规则为 `utf8mb4_0900_ai_ci`。

`utf8mb4` 的意义：支持完整 Unicode，包括中文和表情符号，比 MySQL 早期 `utf8` 更完整。

```
USE campus_activity_db;
```

切换当前数据库，后续建表、插入数据都在这个库中执行。

3.2 为什么这么做

课程演示经常需要重新跑一遍脚本。直接重置数据库可以避免旧数据影响演示效果。

3.3 替代方法

1. 不删除数据库，只 `DROP TABLE`：能保留数据库配置，但清理不彻底。
2. 使用迁移脚本逐步升级：适合生产系统，但答辩初始化较麻烦。
3. 使用 Docker 初始化 SQL：更标准，但环境要求更高。

4. 01_create_database.sql：只创建数据库

核心代码：

```
CREATE DATABASE IF NOT EXISTS campus_activity_db
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_0900_ai_ci;
```

```
USE campus_activity_db;
```

它和 `00_reset_database.sql` 的区别是不会删除已有数据库。适合第一次创建或不想清空数据时使用。

答辩可说：`00` 是全量重置，`01` 是保守创建；实际一键初始化用的是 `00`，因为演示需要稳定初始状态。

5. `02_create_tables.sql`: 核心表设计

这个文件是数据库核心，包含：

1. 删除旧表
2. 创建用户与角色表
3. 创建活动与报名签到表
4. 创建物资借还表
5. 创建处罚和审计表
6. 创建索引

5.1 开头清理旧表

```
USE campus_activity_db;

SET FOREIGN_KEY_CHECKS = 0;

DROP TABLE IF EXISTS audit_log;
DROP TABLE IF EXISTS penalty;
DROP TABLE IF EXISTS borrow_detail;
DROP TABLE IF EXISTS borrow_order;
DROP TABLE IF EXISTS resource_item;
DROP TABLE IF EXISTS checkin;
DROP TABLE IF EXISTS activity_registration;
DROP TABLE IF EXISTS activity;
DROP TABLE IF EXISTS venue;
DROP TABLE IF EXISTS student;
DROP TABLE IF EXISTS club;
DROP TABLE IF EXISTS user_account;

SET FOREIGN_KEY_CHECKS = 1;
```

`SET FOREIGN_KEY_CHECKS = 0` 临时关闭外键检查。因为表之间有外键依赖，如果按错误顺序删表会失败。

删除顺序大体是从业务子表到主表：

1. `audit_log`、`penalty` 是依赖很多业务表的记录表。
2. `borrow_detail` 依赖 `borrow_order` 和 `resource_item`。
3. `borrow_order` 依赖 `activity` 和 `user_account`。
4. `checkin` 依赖 `activity_registration`。
5. `activity_registration` 依赖 `activity` 和 `student`。

6. `activity` 依赖 `club` 和 `venue`。
7. `student`、`club` 依赖 `user_account`。
8. 最后删除 `user_account`。

为什么最后重新 `SET FOREIGN_KEY_CHECKS = 1`：恢复数据库约束，后面创建表和插入数据时继续保证数据一致性。

替代方法：严格按依赖顺序删除，不关闭外键检查。但项目表多，关闭后更稳。

6. 基础用户表：`user_account`

```
CREATE TABLE user_account (  
  user_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password VARCHAR(100) NOT NULL,  
  role VARCHAR(20) NOT NULL,  
  phone VARCHAR(20) NULL,  
  create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CHECK (role IN ('admin', 'club', 'student'))  
);
```

6.1 字段解释

`user_id BIGINT PRIMARY KEY AUTO_INCREMENT`

用户主键，自增。用 `BIGINT` 是为了容量更大，虽然课程项目用 `INT` 也够。

`username VARCHAR(50) NOT NULL UNIQUE`

用户名不能为空且唯一。登录时通过用户名查账号。

`password VARCHAR(100) NOT NULL`

密码字段。当前项目课程演示使用明文密码。

`role VARCHAR(20) NOT NULL`

用户角色，控制权限。

`phone VARCHAR(20) NULL`

手机号，可选。

`create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP`

创建时间，默认当前时间。

`CHECK (role IN ('admin', 'club', 'student'))`

限制角色只能是管理员、社团、学生三类，防止插入非法角色。

6.2 全局作用

`user_account` 是统一账号体系。登录、发帖、交易、借用、社团负责人、学生信息都从这个表关联。

6.3 为什么这么设计

把登录账号单独抽出来，可以让不同业务模块共用同一套账号，不需要活动、交易、社区各自维护用户。

6.4 替代方案

1. 用整数枚举角色：查询效率略好，但可读性差。
2. 单独建 `role` 表：更规范，适合角色很多的系统；本项目只有三类角色，用 `CHECK` 更简单。
3. 密码哈希存储：生产环境必须用 `bcrypt/argon2`；课程演示为了简单用了明文。

7. 社团表： `club`

```
CREATE TABLE club (  
  club_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  club_name VARCHAR(100) NOT NULL UNIQUE,  
  president_user_id BIGINT NULL,  
  contact_phone VARCHAR(20) NULL,  
  office_location VARCHAR(100) NULL,  
  CONSTRAINT fk_club_president  
    FOREIGN KEY (president_user_id) REFERENCES user_account(user_id)  
);
```

7.1 字段解释

`club_id` 是社团主键。

`club_name` 是社团名，唯一，避免两个同名社团。

`president_user_id` 指向 `user_account.user_id`，表示哪个账号是这个社团负责人。

`contact_phone` 和 `office_location` 是社团联系方式和办公地点。

`fk_club_president` 是外键，保证社团负责人必须是一个真实存在的用户账号。

7.2 全局作用

活动由社团发布，物资也归属社团。`activity.club_id` 和 `resource_item.owner_club_id` 都会关联到 `club`。

7.3 为什么 `president_user_id` 允许为空

因为社团资料可能先建，负责人账号可以后绑定。这样初始化或维护时更灵活。

7.4 替代方案

1. 建 `club_member` 表：可以支持一个社团多个管理员。当前项目只需要负责人，未扩展成员关系。

2. `president_user_id NOT NULL`：约束更强，但初始化数据时要求更高。

8. 学生表：student

```
CREATE TABLE student (  
  student_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  user_id BIGINT NOT NULL UNIQUE,  
  student_no VARCHAR(30) NOT NULL UNIQUE,  
  real_name VARCHAR(50) NOT NULL,  
  grade VARCHAR(20) NULL,  
  major VARCHAR(50) NULL,  
  points INT NOT NULL DEFAULT 0,  
  violation_count INT NOT NULL DEFAULT 0,  
  credit_score INT NOT NULL DEFAULT 100,  
  CONSTRAINT fk_student_user  
    FOREIGN KEY (user_id) REFERENCES user_account(user_id),  
  CHECK (credit_score BETWEEN 0 AND 120)  
);
```

8.1 字段解释

`student_id` 是学生业务主键，不直接使用 `user_id` 做主键，可以让账号体系和学生资料解耦。

`user_id BIGINT NOT NULL UNIQUE` 表示一个账号最多对应一个学生资料。

`student_no` 是学号，唯一。

`real_name` 是真实姓名。

`grade` 和 `major` 是年级和专业。

`points` 是积分，活动结算时签到学生会增加。

`violation_count` 是违规次数，缺席活动时会增加。

`credit_score` 是信用分，默认 100，活动缺席或借用逾期会扣分，签到可略微加分。

`CHECK (credit_score BETWEEN 0 AND 120)` 限制信用分范围。

8.2 全局作用

学生表支撑：

1. 活动报名。
2. 活动签到统计。
3. 缺席处罚。
4. 热门活动信用限制。
5. 个人中心展示报名记录。

8.3 为什么不直接把学生字段放进 user_account

因为不是所有用户都是学生。管理员和社团账号不需要学号、专业、信用分。拆表符合不同角色不同扩展信息的设计。

8.4 替代方案

1. 单表继承：所有角色信息都放在 `user_account`，简单但字段大量为空。
2. 多角色详情表：如 `admin_profile`、`club_profile`、`student_profile`，更完整但复杂度高。

9. 场地表：venue

```
CREATE TABLE venue (  
  venue_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  venue_name VARCHAR(100) NOT NULL,  
  capacity INT NOT NULL,  
  location VARCHAR(100) NULL,  
  status VARCHAR(20) NOT NULL DEFAULT 'available',  
  CHECK (capacity > 0),  
  CHECK (status IN ('available', 'unavailable'))  
);
```

9.1 字段解释

`venue_id` 是场地主键。

`venue_name` 是场地名。

`capacity` 是场地容量，必须大于 0。

`location` 是位置。

`status` 表示场地是否可用。

9.2 全局作用

活动必须选择一个场地，`activity.venue_id` 外键关联到这里。统计视图也会展示场地名称。

9.3 设计取舍

当前项目没有做场地时间冲突检测。也就是说，同一场地同一时间可能创建多个活动，这是一个可以被老师追问的点。

替代方案：

1. 增加触发器检查同一场地时间段是否重叠。
2. 单独建 `venue_booking` 表做预约。
3. 在后端创建活动查询冲突。

10. 活动表：activity


```
CREATE TABLE activity (  
  activity_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  club_id BIGINT NOT NULL,  
  venue_id BIGINT NOT NULL,  
  title VARCHAR(200) NOT NULL,  
  category VARCHAR(50) NULL,  
  start_time DATETIME NOT NULL,  
  end_time DATETIME NOT NULL,  
  signup_deadline DATETIME NOT NULL,  
  max_capacity INT NOT NULL,  
  status VARCHAR(20) NOT NULL,  
  description TEXT NULL,  
  CONSTRAINT fk_activity_club  
    FOREIGN KEY (club_id) REFERENCES club(club_id),  
  CONSTRAINT fk_activity_venue  
    FOREIGN KEY (venue_id) REFERENCES venue(venue_id),  
  CHECK (end_time > start_time),  
  CHECK (signup_deadline <= start_time),  
  CHECK (max_capacity > 0),  
  CHECK (status IN ('draft', 'published', 'ongoing', 'finished', 'cancelled'))  
);
```

10.1 字段解释

`club_id` 表示活动归属哪个社团。

`venue_id` 表示活动在哪个场地举行。

`title` 是活动标题。

`category` 是活动分类，例如 `culture`、`training`、`hot`。触发器中对 `hot` 活动有信用分限制。

`start_time`、`end_time` 是活动时间。

`signup_deadline` 是报名截止时间。

`max_capacity` 是最大确认报名人数。

`status` 是活动状态：草稿、已发布、进行中、已结束、已取消。

`description` 是活动描述。

10.2 关键约束

`CHECK (end_time > start_time)` 防止结束时间早于开始时间。

`CHECK (signup_deadline <= start_time)` 防止报名截止晚于活动开始。

`CHECK (max_capacity > 0)` 防止容量为 0 或负数。

`CHECK (status IN (...))` 控制状态枚举。

10.3 全局作用

活动是系统主业务之一。报名、签到、借用单、处罚、统计视图都围绕活动展开。

10.4 为什么状态用字符串

字符串可读性强，页面显示、调试和答解释都方便。

替代方法：

1. 状态用数字编码：更节省空间，但可读性下降。
2. 单独建 `activity_status` 字典表：更规范，但本项目状态固定，没必要增加复杂度。

11. 活动报名表： `activity_registration`

```
CREATE TABLE activity_registration (  
  reg_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  activity_id BIGINT NOT NULL,  
  student_id BIGINT NOT NULL,  
  register_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  audit_status VARCHAR(20) NOT NULL DEFAULT 'approved',  
  checkin_status VARCHAR(20) NOT NULL DEFAULT 'not_checked',  
  reg_status VARCHAR(20) NOT NULL DEFAULT 'confirmed',  
  queue_no INT NULL,  
  CONSTRAINT fk_registration_activity  
    FOREIGN KEY (activity_id) REFERENCES activity(activity_id),  
  CONSTRAINT fk_registration_student  
    FOREIGN KEY (student_id) REFERENCES student(student_id),  
  UNIQUE KEY uq_activity_student (activity_id, student_id),  
  CHECK (audit_status IN ('approved', 'rejected', 'pending')),  
  CHECK (checkin_status IN ('not_checked', 'checked', 'absent')),  
  CHECK (reg_status IN ('confirmed', 'waiting', 'cancelled')),  
  CHECK (queue_no IS NULL OR queue_no > 0)  
);
```

11.1 字段解释

`reg_id` 是报名记录主键。

`activity_id` 指向活动。

`student_id` 指向学生。

`register_time` 是报名时间。

`audit_status` 是审核状态，当前默认 `approved`，预留给未来审批。

`checkin_status` 是签到状态，默认未签到。

`reg_status` 是报名状态：确认、候补、取消。

`queue_no` 是候补序号，只有候补记录需要。

11.2 唯一约束

`UNIQUE KEY uq_activity_student (activity_id, student_id)` 保证同一个学生不能重复报名同一个活动。

这和触发器里的重复检查有重叠。为什么两者都要？

1. 触发器能给更友好的错误信息。
2. 唯一约束是最终兜底，防止并发或遗漏导致重复数据。

11.3 全局作用

这个表连接活动和学生，是报名、候补、签到、结算、统计的核心表。

11.4 替代方案

1. 拆成 `registration` 和 `waitlist` 两张表：候补逻辑更清晰，但查询和转正更复杂。
2. 不做候补，满员直接拒绝：简单，但业务完整度降低。

12. 签到表： `checkin`

```
CREATE TABLE checkin (  
  checkin_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  reg_id BIGINT NOT NULL,  
  checkin_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  operator_id BIGINT NOT NULL,  
  status VARCHAR(20) NOT NULL,  
  CONSTRAINT fk_checkin_reg  
    FOREIGN KEY (reg_id) REFERENCES activity_registration(reg_id),  
  CONSTRAINT fk_checkin_operator  
    FOREIGN KEY (operator_id) REFERENCES user_account(user_id),  
  CHECK (status IN ('checked', 'absent'))  
);
```

12.1 字段解释

`reg_id` 指向报名记录，而不是直接指向学生和活动。因为签到是基于一次报名发生的。

`operator_id` 是操作签到的人，一般是社团账号或管理员账号。

`status` 是签到结果，checked 或 absent。

12.2 为什么报名表里已有 `checkin_status`，还要单独建 `checkin`

`activity_registration.checkin_status` 是当前状态，方便查询。

`checkin` 表是签到操作记录，能保存操作时间和操作人，更适合审计。

这种设计是“状态字段 + 操作流水”的组合。

替代方案：

1. 只保留 `checkin_status`：简单，但无法记录操作人和时间。
2. 只保留 `checkin` 表：更规范，但列表查询要多一次 JOIN。

13. 物资表：resource_item

```
CREATE TABLE resource_item (  
  resource_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  owner_club_id BIGINT NOT NULL,  
  item_name VARCHAR(100) NOT NULL,  
  category VARCHAR(50) NULL,  
  total_qty INT NOT NULL,  
  available_qty INT NOT NULL,  
  unit VARCHAR(20) NOT NULL DEFAULT 'piece',  
  deposit_amount DECIMAL(10, 2) NOT NULL DEFAULT 0,  
  status VARCHAR(20) NOT NULL DEFAULT 'available',  
  CONSTRAINT fk_resource_owner_club  
    FOREIGN KEY (owner_club_id) REFERENCES club(club_id),  
  CHECK (total_qty >= 0),  
  CHECK (available_qty >= 0),  
  CHECK (available_qty <= total_qty),  
  CHECK (status IN ('available', 'unavailable'))  
);
```

13.1 字段解释

`owner_club_id` 表示物资属于哪个社团。

`total_qty` 是总库存。

`available_qty` 是可用库存。

`deposit_amount` 是押金金额，当前页面展示较少，但数据库预留。

13.2 为什么有总量和可用量两个字段

总量表示物资资产总数，可用量表示当前还能借出的数量。借出时减少 `available_qty`，归还时增加 `available_qty`，总量一般不变。

13.3 替代方案

1. 不存 `available_qty`，每次通过借用明细动态计算：更范式化，但查询复杂、性能较差。
2. 单独做库存流水表：更专业，可追踪每次库存变化，但开发量更大。

14. 借用单主表：borrow_order

```
CREATE TABLE borrow_order (  
  order_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  order_uid VARCHAR(32) NULL UNIQUE,  
  activity_id BIGINT NOT NULL,  
  applicant_user_id BIGINT NOT NULL,  
  apply_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  expected_return_time DATETIME NOT NULL,  
  actual_return_time DATETIME NULL,  
  order_status VARCHAR(20) NOT NULL,  
  CONSTRAINT fk_borrow_activity  
    FOREIGN KEY (activity_id) REFERENCES activity(activity_id),  
  CONSTRAINT fk_borrow_applicant  
    FOREIGN KEY (applicant_user_id) REFERENCES user_account(user_id),  
  CHECK (order_status IN ('pending', 'approved', 'borrowed', 'returned',  
    'overdue'))  
);
```

14.1 字段解释

`order_id` 是内部主键。

`order_uid` 是展示给用户的借用单编号，例如 BOR-20260416-000123。

`activity_id` 表示借用物资服务于哪个活动。

`applicant_user_id` 是申请人账号。

`expected_return_time` 是应归还时间。

`actual_return_time` 是实际归还时间。

`order_status` 是借用单状态。

14.2 为什么同时有 `order_id` 和 `order_uid`

`order_id` 适合数据库内部关联，短、稳定、自增。

`order_uid` 适合页面展示和人工查找，包含日期和编号，更有业务含义。

14.3 替代方案

1. 只用 `order_id`：简单，但用户体验差。
2. 用 UUID：全局唯一，但不如当前格式可读。
3. 用数据库触发器自动生成 `order_uid`：可以，但需要处理自增 ID 获取问题，后端生成更直观。

15. 借用明细表： `borrow_detail`

```
CREATE TABLE borrow_detail (  
  detail_id BIGINT PRIMARY KEY AUTO_INCREMENT,
```

```
order_id BIGINT NOT NULL,
resource_id BIGINT NOT NULL,
borrow_qty INT NOT NULL,
returned_qty INT NOT NULL DEFAULT 0,
damage_qty INT NOT NULL DEFAULT 0,
compensation_amount DECIMAL(10, 2) NOT NULL DEFAULT 0,
CONSTRAINT fk_borrow_detail_order
    FOREIGN KEY (order_id) REFERENCES borrow_order(order_id),
CONSTRAINT fk_borrow_detail_resource
    FOREIGN KEY (resource_id) REFERENCES resource_item(resource_id),
CHECK (borrow_qty > 0),
CHECK (returned_qty >= 0),
CHECK (damage_qty >= 0),
CHECK (returned_qty <= borrow_qty),
CHECK (damage_qty <= borrow_qty)
);
```

15.1 字段解释

一张借用单可以包含多个物资，所以需要明细表。

`borrow_qty` 是借出数量。

`returned_qty` 是已归还数量。

`damage_qty` 是损坏数量。

`compensation_amount` 是赔偿金额。

15.2 为什么主从表设计

一张订单可能借多个物资，比如音箱 2 套、话筒 4 个。主表保存订单整体信息，明细表保存每种物资的数量。

替代方案：

1. 一个借用单只能借一种物资：设计简单，但业务不自然。
2. 明细中冗余物资名称：减少 JOIN，但数据容易不一致。

16. 处罚表：penalty

```
CREATE TABLE penalty (
    penalty_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    student_id BIGINT NULL,
    activity_id BIGINT NULL,
    order_id BIGINT NULL,
    penalty_type VARCHAR(30) NOT NULL,
    reason VARCHAR(200) NOT NULL,
    amount DECIMAL(10, 2) NOT NULL DEFAULT 0,
    status VARCHAR(20) NOT NULL DEFAULT 'unpaid',
    create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_penalty_student
```

```
FOREIGN KEY (student_id) REFERENCES student(student_id),
CONSTRAINT fk_penalty_activity
FOREIGN KEY (activity_id) REFERENCES activity(activity_id),
CONSTRAINT fk_penalty_order
FOREIGN KEY (order_id) REFERENCES borrow_order(order_id),
CHECK (penalty_type IN ('no_show', 'overdue', 'damage')),
CHECK (status IN ('unpaid', 'paid', 'closed'))
);
```

16.1 字段解释

处罚可能来源于活动缺席，也可能来源于借用逾期或损坏。

`student_id` 表示被处罚学生。

`activity_id` 表示关联活动。

`order_id` 表示关联借用单。

`penalty_type` 区分处罚类型。

`amount` 是金额，当前缺席和逾期默认 0，损坏可扩展赔偿。

16.2 为什么多个外键允许为空

不同处罚场景不一定都有所有关联。例如缺席处罚有学生和活动，不一定有借用单；逾期处罚可能有关联借用单。

替代方案：

1. 分成 `activity_penalty`、`borrow_penalty` 两张表：字段更明确，但查询总处罚时麻烦。
2. 用通用业务类型和业务 ID：更灵活，但外键无法直接约束。

17. 审计日志表： `audit_log`

```
CREATE TABLE audit_log (
  log_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  biz_type VARCHAR(30) NOT NULL,
  biz_id BIGINT NOT NULL,
  action VARCHAR(30) NOT NULL,
  operator_id BIGINT NULL,
  old_data JSON NULL,
  new_data JSON NULL,
  op_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_audit_operator
  FOREIGN KEY (operator_id) REFERENCES user_account(user_id)
);
```

17.1 字段解释

`biz_type` 表示业务类型，如 `activity`、`borrow_order`。

`biz_id` 表示对应业务记录 ID。

`action` 表示动作，如 `status_change`。

`old_data` 和 `new_data` 用 JSON 保存变化前后的数据。

17.2 为什么用 JSON

不同业务对象字段不同，用 JSON 可以灵活保存变化内容，不需要为每种日志建一张表。

替代方案：

1. 每个业务单独一张日志表：结构强，但表多。
2. 只保存文本描述：简单，但后续机器分析困难。
3. 应用层写日志：更容易记录当前操作者，但数据库外部操作可能漏记。

18. 索引设计

```
CREATE INDEX idx_activity_status_start
ON activity(status, start_time);

CREATE INDEX idx_activity_registration_activity_status
ON activity_registration(activity_id, reg_status);

CREATE INDEX idx_checkin_reg_status
ON checkin(reg_id, status);

CREATE INDEX idx_borrow_order_status_expected
ON borrow_order(order_status, expected_return_time);

CREATE INDEX idx_borrow_detail_order_resource
ON borrow_detail(order_id, resource_id);

CREATE INDEX idx_penalty_student_status
ON penalty(student_id, status);
```

18.1 每个索引为什么存在

`idx_activity_status_start`

活动列表常按状态筛选并按开始时间排序，例如优先显示 `published`、`ongoing` 活动。

`idx_activity_registration_activity_status`

活动详情和统计经常按 `activity_id` 查询报名记录，并统计 `confirmed/waiting`。

`idx_checkin_reg_status`

签到查询按报名记录和状态查找。

`idx_borrow_order_status_expected`

逾期事件需要查 `order_status='borrowed' AND expected_return_time < NOW()`。

`idx_borrow_detail_order_resource`

归还和统计时经常按订单查明细，或关联物资。

`idx_penalty_student_status`

个人中心或处罚管理可以按学生和处罚状态查询。

18.2 为什么在建太多索引

索引能提高查询速度，但会降低插入和更新性能，并占用存储空间。课程项目只给高频查询字段建索引。

替代方案：根据真实慢查询日志继续优化索引。

19. 03_insert_init_data.sql：初始数据

这个文件用于快速演示。

19.1 用户数据

插入 7 个账号：

1. `admin1`：管理员。
2. `club_music`：音乐社团负责人。
3. `club_volunteer`：志愿者社团负责人。
4. `stu_zhang`、`stu_li`、`stu_wang`、`stu_zhao`：学生。

密码均为 `123456`。

19.2 社团和学生数据

`club` 中插入音乐社、志愿者社，并把负责人关联到账号 2 和 3。

`student` 中插入 4 个学生，并把学生资料关联到账号 4 到 7。

19.3 场地数据

插入三个场地：教室、操场、讲座室。

19.4 活动数据

插入四个活动：

1. `Campus Band Night`：已发布，容量 2，用于演示报名和候补。
2. `Weekend Cleaning Action`：已发布，容量 100。
3. `Old Activity For Finish Demo`：ongoing，结束时间已经过去，用于演示结算过程。
4. `Cancelled Activity For Delete Demo`：cancelled，用于演示事务删除。

19.5 物资和借用数据

插入音箱、话筒、垃圾夹等物资。

再插入一个借用单和两条借用明细，并手动扣减库存。

19.6 报名和签到数据

活动 1 容量为 2，初始化两个 confirmed、一个 waiting，用于演示候补。

活动 3 插入一个 checked、一个 absent，用于演示结算时积分、信用分和处罚变化。

19.7 为什么要有初始化数据

没有演示数据，答辩时每个功能都要现场创建，耗时且容易出错。初始化数据可以让关键功能一打开就能演示。

替代方案：

1. 页面提供“生成演示数据”按钮。
2. 使用 Faker 随机生成大量数据。
3. 用测试脚本插入数据。

20. 04_trigger.sql：触发器

触发器是数据库课程展示重点之一。本项目有三个触发器。

20.1 trg_before_insert_registration

触发时机：

```
BEFORE INSERT ON activity_registration
FOR EACH ROW
```

意思是：每插入一条报名记录之前，数据库都会自动执行这段逻辑。

20.1.1 声明变量

```
DECLARE v_status VARCHAR(20);
DECLARE v_deadline DATETIME;
DECLARE v_capacity INT;
DECLARE v_confirmed_count INT DEFAULT 0;
DECLARE v_waiting_max INT;
DECLARE v_credit INT;
DECLARE v_category VARCHAR(50);
```

这些变量用于临时保存活动状态、报名截止、容量、已确认人数、最大候补号、学生信用分、活动分类。

20.1.2 查询活动信息

```
SELECT status, signup_deadline, max_capacity, category
  INTO v_status, v_deadline, v_capacity, v_category
FROM activity
WHERE activity_id = NEW.activity_id;
```

`NEW.activity_id` 表示即将插入的新报名记录中的活动 ID。

这句代码把对应活动的信息查出来，供后续判断。

20.1.3 检查活动是否存在

```
IF v_status IS NULL THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Activity does not exist';
END IF;
```

如果没查到活动，就抛出异常。`SIGNAL SQLSTATE '45000'` 是 MySQL 中主动抛业务错误的方式。

注意：由于 `activity_id` 有外键，活动不存在本来也会失败。这里的意义是给更明确的业务错误。

20.1.4 检查活动必须已发布

```
IF v_status <> 'published' THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Activity is not published';
END IF;
```

只有 `published` 活动能报名。`draft`、`ongoing`、`finished`、`cancelled` 都不允许报名。

20.1.5 检查报名截止时间

```
IF NOW() > v_deadline THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Signup deadline has passed';
END IF;
```

数据库以当前时间判断是否超过报名截止。

20.1.6 检查重复报名

```
IF EXISTS (
  SELECT 1
  FROM activity_registration
```

```
WHERE activity_id = NEW.activity_id
AND student_id = NEW.student_id
) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Duplicate registration is not
allowed';
END IF;
```

如果该学生已经报名这个活动，抛错。

20.1.7 信用分限制

```
SELECT credit_score INTO v_credit
FROM student
WHERE student_id = NEW.student_id;

IF v_category = 'hot' AND v_credit < 60 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Credit score too low for hot
activity';
END IF;
```

对于分类为 **hot** 的热门活动，信用分低于 60 的学生不能报名。

20.1.8 容量和候补逻辑

```
SELECT COUNT(*) INTO v_confirmed_count
FROM activity_registration
WHERE activity_id = NEW.activity_id
AND reg_status = 'confirmed';
```

先统计当前 confirmed 人数。

如果没满：

```
IF v_confirmed_count < v_capacity THEN
    SET NEW.reg_status = 'confirmed';
    SET NEW.queue_no = NULL;
```

直接把新记录改成 confirmed。

如果满了：

```
ELSE
    SET NEW.reg_status = 'waiting';

    SELECT COALESCE(MAX(queue_no), 0)
```

```
    INTO v_waiting_max
  FROM activity_registration
 WHERE activity_id = NEW.activity_id
       AND reg_status = 'waiting';

  SET NEW.queue_no = v_waiting_max + 1;
END IF;
```

设置为 waiting，并把候补序号设为当前最大候补号 + 1。

20.1.9 为什么报名规则放在触发器里

如果只在 Flask 后端判断，其他入口直接写数据库可能绕过规则。触发器在数据库层执行，任何 **INSERT INTO activity_registration** 都必须经过它。

20.1.10 替代方案

1. 后端代码判断：更容易调试，但不能防止绕过应用层。
2. 存储过程报名：让所有报名调用 **CALL sp_register_activity**，逻辑更清楚，但需要强制所有入口使用过程。
3. 数据库约束：能处理唯一性和枚举，但无法处理容量、截止时间、信用分这种复杂逻辑。

20.2 trg_after_update_borrow_order_audit

触发时机：

```
AFTER UPDATE ON borrow_order
FOR EACH ROW
```

每次更新借用单后执行。

核心逻辑：

```
IF OLD.order_status <> NEW.order_status THEN
  INSERT INTO audit_log(...)
  VALUES (...);
END IF;
```

OLD 表示更新前数据，**NEW** 表示更新后数据。

只有状态变化时才写日志，避免普通字段更新产生无意义日志。

日志中：

1. **biz_type** 是 **borrow_order**。
2. **biz_id** 是借用单 ID。

3. `action` 是 `status_change`。
4. `operator_id` 当前用 `NEW.applicant_user_id`。
5. `old_data` 和 `new_data` 用 JSON 保存旧状态、新状态和归还时间。

为什么这么做：借用单状态变化是关键操作，应该留痕，方便审计。

替代方案：

1. 后端每次状态变更时手动写日志。
2. 使用 MySQL binlog 审计。
3. 单独建立借用状态历史表。

20.3 `trg_after_update_activity_audit`

和借用单审计类似，只是针对活动表。

当 `activity.status` 改变时，写入 `audit_log`：

```
'activity',
NEW.activity_id,
'status_change',
NULL,
JSON_OBJECT('status', OLD.status),
JSON_OBJECT('status', NEW.status)
```

`operator_id` 这里为 `NULL`，因为数据库触发器不知道当前 Flask 登录用户是谁。

答辩可以主动说：如果要更精确记录操作者，可以由应用层写审计日志，或使用数据库会话变量传入当前用户 ID。

21. `05_procedure.sql`：存储过程

存储过程用于封装复杂数据库业务逻辑。本项目有三个过程。

21.1 `sp_promote_waitlist`

作用：把候补队列第一名转正。

21.1.1 开启事务

```
START TRANSACTION;
```

候补转正和队列重排必须一起成功或一起失败。

21.1.2 查询候补第一名并加锁

```
SELECT reg_id
      INTO v_reg_id
FROM activity_registration
WHERE activity_id = p_activity_id
      AND reg_status = 'waiting'
ORDER BY queue_no ASC
LIMIT 1
FOR UPDATE;
```

FOR UPDATE 锁住被查询的候补记录，防止并发同时转正同一个人。

21.1.3 没有候补则结束

```
IF v_reg_id IS NULL THEN
  COMMIT;
  LEAVE proc_promote;
END IF;
```

如果没有候补，直接提交并退出。

21.1.4 转正

```
UPDATE activity_registration
SET reg_status = 'confirmed', queue_no = NULL
WHERE reg_id = v_reg_id;
```

候补第一名变 confirmed，候补号清空。

21.1.5 重排候补队列

```
SET @rownum := 0;
UPDATE activity_registration ar
JOIN (
  SELECT reg_id, (@rownum := @rownum + 1) AS new_queue_no
  FROM activity_registration
  WHERE activity_id = p_activity_id AND reg_status = 'waiting'
  ORDER BY queue_no ASC, reg_id ASC
) q ON ar.reg_id = q.reg_id
SET ar.queue_no = q.new_queue_no;
```

把剩余候补重新编号为 1、2、3，避免出现 2、3、4 这种空洞。

21.1.6 当前项目中的状态

该过程已经在数据库层实现，但后端当前没有提供“取消报名后自动候补转正”的入口调用它。答辩时可以说它是预留能力。

21.1.7 替代方案

1. 后端查询候补第一名并更新。
2. 在取消报名触发器中自动调用候补转正逻辑。
3. 不重排候补号，只按原始号排序。

21.2 `sp_finish_activity`

作用：活动结算。

这是项目最适合答辩展示的存储过程。

21.2.1 开启事务

```
START TRANSACTION;
```

结算会修改活动、学生、处罚、借用单等多张表，必须保证原子性。

21.2.2 查询并锁定活动

```
SELECT status, end_time
  INTO v_status, v_end_time
FROM activity
WHERE activity_id = p_activity_id
FOR UPDATE;
```

`FOR UPDATE` 防止多个管理员同时结算同一活动。

21.2.3 校验活动存在

```
IF v_status IS NULL THEN
  ROLLBACK;
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Activity not found';
END IF;
```

没有活动则回滚并抛错。

21.2.4 校验活动状态


```
IF v_status <> 'ongoing' THEN
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only ongoing activity can be
finished';
END IF;
```

只有 ongoing 活动能结算，避免重复结算 finished 活动。

21.2.5 校验结束时间

```
IF NOW() < v_end_time THEN
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Activity has not reached end_time
yet';
END IF;
```

活动还没结束不能结算。

21.2.6 修改活动状态

```
UPDATE activity
SET status = 'finished'
WHERE activity_id = p_activity_id;
```

活动变为 finished。这个更新也会触发活动审计触发器。

21.2.7 奖励签到学生

```
UPDATE student s
JOIN activity_registration ar ON ar.student_id = s.student_id
SET s.points = s.points + 5,
    s.credit_score = LEAST(120, s.credit_score + 1)
WHERE ar.activity_id = p_activity_id
    AND ar.reg_status = 'confirmed'
    AND ar.checkin_status = 'checked';
```

对 confirmed 且 checked 的学生：

1. 积分加 5。
2. 信用分加 1，但最高不超过 120。

`LEAST(120, ...)` 用于上限保护。

21.2.8 惩罚缺席学生

```
UPDATE student s
JOIN activity_registration ar ON ar.student_id = s.student_id
SET s.violation_count = s.violation_count + 1,
    s.credit_score = GREATEST(0, s.credit_score - 5)
WHERE ar.activity_id = p_activity_id
      AND ar.reg_status = 'confirmed'
      AND ar.checkin_status <> 'checked';
```

对 confirmed 但没有 checked 的学生：

1. 违规次数加 1。
2. 信用分扣 5，但最低不低于 0。

`GREATEST(0, ...)` 用于下限保护。

21.2.9 生成缺席处罚

```
INSERT INTO penalty(student_id, activity_id, penalty_type, reason, amount, status)
SELECT ar.student_id,
       p_activity_id,
       'no_show',
       'Absent after successful registration',
       0,
       'closed'
FROM activity_registration ar
WHERE ar.activity_id = p_activity_id
      AND ar.reg_status = 'confirmed'
      AND ar.checkin_status <> 'checked';
```

把缺席学生批量插入处罚表。

这里使用 `INSERT INTO ... SELECT`，好处是不用后端循环逐条插入。

21.2.10 标记逾期借用单

```
UPDATE borrow_order bo
SET bo.order_status = 'overdue'
WHERE bo.activity_id = p_activity_id
      AND bo.order_status = 'borrowed'
      AND bo.expected_return_time < NOW();
```

当前活动相关的已借出且超过应归还时间的借用单改为 overdue。

21.2.11 给逾期借用生成处罚

```
INSERT INTO penalty(student_id, activity_id, order_id, penalty_type, reason,
amount, status)
SELECT DISTINCT st.student_id,
                bo.activity_id,
                bo.order_id,
                'overdue',
                'Resource overdue return',
                0,
                'unpaid'
FROM borrow_order bo
JOIN user_account ua ON ua.user_id = bo.applicant_user_id
JOIN student st ON st.user_id = ua.user_id
WHERE bo.activity_id = p_activity_id
      AND bo.order_status = 'overdue';
```

这里通过借用单申请人账号找到学生资料。只有学生申请人才会生成学生处罚。

21.2.12 扣逾期学生信用分

```
UPDATE student s
JOIN user_account ua ON ua.user_id = s.user_id
JOIN borrow_order bo ON bo.applicant_user_id = ua.user_id
SET s.credit_score = GREATEST(0, s.credit_score - 8)
WHERE bo.activity_id = p_activity_id
      AND bo.order_status = 'overdue';
```

逾期扣 8 分。

21.2.13 提交事务

```
COMMIT;
```

所有步骤成功后提交。

21.2.14 为什么结算放在存储过程

活动结算涉及多个表，逻辑强依赖数据库数据。放在存储过程里可以：

1. 减少后端多次往返数据库。
2. 保证事务边界集中。
3. 便于课程展示数据库编程能力。
4. 避免应用层部分更新成功、部分失败。

21.2.15 替代方案

1. 后端 Python 中用事务执行多条 SQL：更容易调试，但数据库展示能力弱。
2. 定时任务自动结算活动：更自动化，但课程演示不够可控。
3. 触发器自动结算：不适合，因为结算不是单表简单变更，而是一个业务动作。

21.3 `sp_delete_cancelled_activity`

作用：事务性删除已取消活动。

21.3.1 为什么只能删除 `cancelled` 活动

如果活动还在 `published` 或 `ongoing`，直接删除会破坏报名、借用、统计数据。只允许删除 `cancelled` 活动能降低误删风险。

21.3.2 查询并锁定活动

```
SELECT status INTO v_status
FROM activity
WHERE activity_id = p_activity_id
FOR UPDATE;
```

锁住活动，防止删除过程中状态被别人修改。

21.3.3 校验存在和状态

不存在则抛 `Activity not found`。

不是 `cancelled` 则抛 `Only cancelled activity can be deleted`。

21.3.4 删除顺序

```
DELETE c
FROM checkin c
JOIN activity_registration ar ON ar.reg_id = c.reg_id
WHERE ar.activity_id = p_activity_id;
```

先删除签到记录，因为它依赖报名记录。

```
DELETE FROM penalty
WHERE activity_id = p_activity_id
OR order_id IN (
    SELECT order_id FROM borrow_order WHERE activity_id = p_activity_id
);
```

删除相关处罚。

```
DELETE bd
FROM borrow_detail bd
JOIN borrow_order bo ON bo.order_id = bd.order_id
WHERE bo.activity_id = p_activity_id;
```

删除借用明细。

```
DELETE FROM borrow_order
WHERE activity_id = p_activity_id;
```

删除借用主单。

```
DELETE FROM activity_registration
WHERE activity_id = p_activity_id;
```

删除报名记录。

```
DELETE FROM activity
WHERE activity_id = p_activity_id;
```

最后删除活动本身。

21.3.5 为什么要手动按顺序删除

因为存在外键依赖。如果先删活动，子表数据还在，会违反外键约束。

21.3.6 替代方案

1. 外键加 **ON DELETE CASCADE**：删除活动时自动删除子表，代码更短，但课程展示事务删除逻辑不明显，且风险较大。
2. 软删除：给活动加 **deleted_at** 或 **status**，不物理删除。生产系统更常用。
3. 后端多条 SQL 删除：也可以，但事务逻辑放在应用层。

22. 06_view.sql：视图

视图是把复杂 SELECT 保存成数据库对象，后端可以像查表一样查它。

22.1 v_activity_summary

作用：活动综合统计。

它查询：

1. 活动基本信息。
2. 社团名称。
3. 场地名称。
4. confirmed 人数。
5. waiting 人数。
6. checked 人数。
7. 借用总数量。

核心结构：

```
FROM activity a
JOIN club c ON c.club_id = a.club_id
JOIN venue v ON v.venue_id = a.venue_id
LEFT JOIN (...) r ON r.activity_id = a.activity_id
LEFT JOIN (...) b ON b.activity_id = a.activity_id;
```

`JOIN club` 和 `JOIN venue` 是必需关联，因为活动必须有社团和场地。

`LEFT JOIN` 报名统计和借用统计，是因为活动可能还没有报名或借用记录，也应该出现在统计里。

`COALESCE(..., 0)` 把空值转为 0，页面显示更友好。

22.2 v_hot_activity_top10

作用：热门活动 Top10。

它基于 `v_activity_summary`，按 confirmed 人数和签到率排序。

签到率计算：

```
CASE WHEN t.confirmed_count = 0 THEN 0
      ELSE t.checked_count / t.confirmed_count * 100
END
```

为什么要判断 `confirmed_count` 为 0：避免除以 0。

`ROUND(..., 2)` 保留两位小数。

`LIMIT 10` 只取前十。

22.3 v_club_resource_utilization

作用：社团物资使用情况统计。

统计：

1. 总借用数量。
2. 总损坏数量。
3. 损坏率。

4. 逾期率。

它从 `club` 左连接 `activity`、`borrow_order`、`borrow_detail`，保证没有借用记录的社团也能显示。

22.4 为什么用视图

1. 后端查询简单，`SELECT * FROM v_activity_summary` 即可。
2. 复杂 JOIN 和聚合逻辑集中在数据库。
3. 答辩能展示数据库视图能力。
4. 统计口径统一，多个页面查询不会写出不同版本。

22.5 替代方案

1. 后端直接写复杂 SELECT：灵活但重复。
2. 建统计表定时刷新：性能更好，但数据实时性差。
3. 使用物化视图：MySQL 原生不直接支持，需要自己维护。

23. 07_event.sql：定时事件

```
SET GLOBAL event_scheduler = ON;

CREATE EVENT ev_mark_overdue_orders
ON SCHEDULE EVERY 30 MINUTE
DO
BEGIN
    ...
END
```

23.1 作用

每 30 分钟自动检查借用单：

1. 如果状态是 `borrowed`。
2. 且 `expected_return_time < NOW()`。
3. 则改为 `overdue`。
4. 并给对应学生插入 `overdue` 处罚。

23.2 防重复处罚

```
AND NOT EXISTS (
    SELECT 1
    FROM penalty p
    WHERE p.order_id = bo.order_id
        AND p.penalty_type = 'overdue'
)
```

这段保证同一借用单不会每 30 分钟重复插入逾期处罚。

23.3 为什么用事件

逾期不是用户点击触发的，而是时间到了自动发生。MySQL Event 可以让数据库自己定期处理。

23.4 替代方案

1. Flask 后台定时任务 APScheduler。
2. Linux cron 定时执行 SQL。
3. 每次用户访问借用页面时顺便检查逾期。

数据库事件的优点是逻辑靠近数据，缺点是需要数据库开启 event scheduler，部署时要注意权限。

24. 08_add_trade_community.sql：二手交易和社区扩展

这个文件把系统从单一活动系统扩展成“校园百事通”。

24.1 二手交易分类表：product_category

字段：

1. category_id：分类主键。
2. category_name：分类名，唯一。
3. status：active/inactive。
4. create_time：创建时间。

作用：商品分类，例如 books、electronics、daily_goods。

为什么分类单独建表：避免商品表里随便写分类字符串，便于统一管理。

24.2 商品表：product

字段：

1. seller_user_id：卖家账号。
2. category_id：商品分类。
3. title、description：商品信息。
4. price DECIMAL(10,2)：价格。
5. status：on_sale、locked、sold、removed。
6. publish_time：发布时间。

约束：

1. 卖家必须是存在的用户。
2. 分类必须存在。
3. 价格不能小于 0。
4. 状态必须合法。

状态设计：

on_sale 表示可买。

`locked` 表示已经有人下单，暂时锁定，防止重复购买。

`sold` 表示交易完成。

`removed` 表示下架。

24.3 二手订单表: `trade_order`

字段:

1. `product_id`: 关联商品。
2. `buyer_user_id`: 买家。
3. `seller_user_id`: 卖家。
4. `order_status`: `created`、`cancelled`、`completed`。
5. `create_time`、`finish_time`: 创建和完成时间。

关键约束:

```
CHECK (buyer_user_id <> seller_user_id)
```

防止自己买自己的商品。

24.4 二手交易索引

`idx_product_status_time`: 商品列表按状态和发布时间查。

`idx_trade_order_buyer_status`: 查询我作为买家的订单。

`idx_trade_order_seller_status`: 查询我作为卖家的订单。

24.5 社区分类表: `post_category`

类似商品分类，用于管理帖子分类，如 `campus_life`、`lost_found`、`experience_share`。

24.6 帖子表: `post`

字段:

1. `author_user_id`: 发帖人。
2. `category_id`: 帖子分类。
3. `title`: 标题。
4. `content`: 正文。
5. `status`: `visible`/`hidden`。
6. `create_time`: 发布时间。

24.7 评论表: `post_comment`

字段:

1. `post_id`: 评论属于哪个帖子。

2. `user_id`: 评论人。
3. `content`: 评论内容, 最多 500 字。
4. `create_time`: 评论时间。

24.8 为什么 08 文件中大量使用 `IF NOT EXISTS`

因为这是增量扩展脚本, 可能在旧数据库上重复执行。`IF NOT EXISTS` 可以避免表已存在时报错。

24.9 为什么用动态 SQL 创建索引

MySQL 的 `CREATE INDEX` 没有所有版本都支持 `IF NOT EXISTS`。所以脚本先查 `information_schema.statistics` 判断索引是否存在, 再通过 `PREPARE/EXECUTE` 动态执行。

24.10 种子分类数据

脚本最后插入默认商品分类和帖子分类, 但只在分类表为空时插入。

这样可以避免重复执行脚本时重复插入分类。

25. 09_add_post_like.sql: 点赞表

```
CREATE TABLE IF NOT EXISTS post_like (  
  like_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  post_id BIGINT NOT NULL,  
  user_id BIGINT NOT NULL,  
  create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_post_like_post  
    FOREIGN KEY (post_id) REFERENCES post(post_id),  
  CONSTRAINT fk_post_like_user  
    FOREIGN KEY (user_id) REFERENCES user_account(user_id),  
  UNIQUE KEY uq_post_like_user (post_id, user_id)  
);
```

25.1 字段解释

`post_id` 表示点赞的是哪个帖子。

`user_id` 表示谁点的赞。

`create_time` 表示点赞时间。

25.2 唯一键作用

`UNIQUE KEY uq_post_like_user (post_id, user_id)` 保证一个用户对同一个帖子只能点赞一次。

取消点赞时删除该行, 再点赞时重新插入。

25.3 替代方案

1. 在 `post` 表里加 `like_count` 字段: 查询快, 但需要维护计数一致性。

2. 使用 Redis 记录点赞：适合高并发，但课程项目太重。
3. 点赞表加 `status` 字段做软删除：可以保留历史，但查询稍复杂。

26. `10_add_borrow_order_uid.sql`：旧库升级脚本

这个文件用于已经存在的旧数据库。

26.1 检查列是否存在

```
SELECT COUNT(1)
FROM information_schema.columns
WHERE table_schema = DATABASE()
      AND table_name = 'borrow_order'
      AND column_name = 'order_uid';
```

如果 `borrow_order` 还没有 `order_uid` 列，就执行 `ALTER TABLE` 添加。

26.2 补历史数据

```
UPDATE borrow_order
SET order_uid = CONCAT('BOR-', DATE_FORMAT(apply_time, '%Y%m%d'), '-',
LPAD(order_id, 6, '0'))
WHERE order_uid IS NULL;
```

根据申请时间和内部 ID 生成 UID。

26.3 为什么需要这个脚本

项目早期可能没有 `order_uid`。如果直接改 `02_create_tables.sql`，新库没问题，但旧库不会自动多出字段。所以提供一个增量升级脚本。

27. 一键初始化脚本

27.1 `99_bootstrap_all.sql`

使用 MySQL CLI 的 `SOURCE` 命令按顺序执行所有脚本：

1. 重置数据库。
2. 建核心表。
3. 插入初始化数据。
4. 创建触发器。
5. 创建存储过程。
6. 创建视图。
7. 创建事件。
8. 创建二手和社区扩展表。

9. 创建点赞表。

注意：`10_add_borrow_order_uid.sql` 注释掉了，因为全量新建时 `02_create_tables.sql` 已经包含 `order_uid`。

27.2 99_bootstrap_workbench.sql

Workbench 对 `SOURCE` 支持不方便，所以这个文件把所有 SQL 展开到一个文件中，适合直接在 Workbench 执行。

28. 数据库与后端代码的对应关系

28.1 数据库连接

`core/db.py`:

```
pymysql.connect(..., cursorclass=pymysql.cursors.DictCursor, autocommit=False)
```

`DictCursor` 让查询结果变成字典，模板可以用 `row["title"]` 或 `row.title` 风格读取。

`autocommit=False` 表示默认不开自动提交。写操作必须显式 `conn.commit()`，失败时 `conn.rollback()`。

为什么这么做：事务需要手动控制，否则多表操作失败时无法整体回滚。

28.2 活动报名对应触发器

`routes/activity_routes.py` 中报名路由只执行：

```
INSERT INTO activity_registration(activity_id, student_id) VALUES (%s, %s)
```

它没有 Python 中判断容量、截止、候补，因为这些由 `trg_before_insert_registration` 统一处理。

如果触发器抛异常，Python 捕获异常并映射成中文提示。

28.3 活动结算对应存储过程

结算路由调用：

```
CALL sp_finish_activity(%s)
```

活动状态、积分、信用分、处罚、逾期借用单都在数据库过程里处理。

28.4 活动删除对应存储过程

删除路由调用：

```
CALL sp_delete_cancelled_activity(%s)
```

这样后端不用自己写一串删除 SQL。

28.5 借用库存并发控制

添加借用明细时，后端执行：

```
SELECT available_qty FROM resource_item WHERE resource_id=%s FOR UPDATE
```

然后检查库存、插入明细、扣减库存。

为什么用 **FOR UPDATE**：如果两个人同时借最后一件物资，不加锁可能都看到库存充足，导致库存扣成负数。加锁后第二个事务必须等第一个事务完成。

28.6 二手下单并发控制

创建订单时：

```
SELECT product_id, seller_user_id, status FROM product WHERE product_id=%s FOR UPDATE
```

锁住商品行，确认商品还是 **on_sale** 后插入订单并改成 **locked**。防止同一个商品被多人同时下单。

29. 数据库部分可以主动说明的优点

1. 约束完整：外键、唯一键、CHECK 覆盖主要非法数据。
2. 业务规则下沉：报名核心规则在触发器中，不依赖前端。
3. 事务明确：结算、删除、借用、归还、交易下单都考虑了原子性。
4. 并发意识：库存扣减和商品下单使用 **FOR UPDATE**。
5. 统计封装：视图减少后端复杂 SQL。
6. 自动化处理：事件定时标记逾期订单。
7. 可演示性强：初始化数据专门覆盖报名、候补、结算、删除、借还。

30. 数据库部分可以主动承认的不足

这些不是致命问题，答辩中主动承认并给改进方案会显得更成熟。

30.1 密码明文存储

当前 **user_account.password** 是明文。生产环境应使用 bcrypt 或 argon2 哈希。

30.2 活动场地没有冲突检测

当前只检查时间顺序，没有检查同一场地同一时间是否被占用。

改进方案：创建活动增加查询，或用触发器检查时间区间重叠。

30.3 候补转正过程未接入前端

数据库有 `sp_promote_waitlist`，但当前没有取消报名入口自动调用它。

改进方案：增加取消报名功能，取消 `confirmed` 报名后调用候补转正过程。

30.4 审计日志操作者不够精确

活动状态触发器的 `operator_id` 是 `NULL`，因为数据库触发器不知道当前登录用户。

改进方案：在后端写审计日志，或设置数据库会话变量传递当前用户。

30.5 部分统计视图没有在页面展示

`v_club_resource_utilization` 已创建，但当前统计页面主要展示活动统计和热门活动。

改进方案：在统计页面增加社团物资利用率板块。

31. 老师可能问的问题与回答思路

问题 1：你这个系统有哪些表？核心关系是什么？

回答思路：

系统基础是 `user_account`，按角色扩展出 `student` 和 `club`。活动由 `club` 发布，在 `venue` 举行。学生通过 `activity_registration` 报名，签到记录在 `checkin`。物资存在 `resource_item`，借用通过 `borrow_order` 和 `borrow_detail` 形成主从关系。处罚统一放在 `penalty`，状态变化写入 `audit_log`。扩展模块中，二手交易是 `product_category/product/trade_order`，社区是 `post_category/post/post_comment/post_like`。

问题 2：为什么要把报名规则写成触发器？

回答思路：

报名涉及活动状态、报名截止、重复报名、信用分、容量、候补序号等规则。如果只写在后端，其他数据库入口可能绕过。触发器在插入报名记录前执行，可以保证任何插入都满足规则。后端只需要处理成功或失败提示。

问题 3：触发器中如何实现候补？

回答思路：

先统计当前 `confirmed` 人数。如果小于活动 `max_capacity`，新报名设为 `confirmed`；否则设为 `waiting`，并查询当前最大 `queue_no`，新记录的候补号为最大值加 1。

问题 4：如何防止重复报名？

回答思路：

有两层保护。第一层是触发器中 `EXISTS` 查询，发现重复报名就 `SIGNAL` 抛错。第二层是 `UNIQUE KEY uq_activity_student(activity_id, student_id)`，数据库唯一约束最终兜底。

问题 5：为什么活动结算用存储过程？

回答思路：

结算不是单表操作，而是同时更新活动状态、学生积分、信用分、违规次数、处罚表和逾期借用单。用存储过程可以把这些操作放在同一个事务里，保证要么全部成功，要么全部失败，同时减少后端代码复杂度。

问题 6： `sp_finish_activity` 如何保证不会提前结算？

回答思路：

过程先查询活动状态和结束时间，并 `FOR UPDATE` 锁定活动行。只有状态是 `ongoing` 且当前时间已经超过 `end_time` 时才允许继续，否则 `ROLLBACK` 并抛出错误。

问题 7：删除活动为什么只能删除 `cancelled`？

回答思路：

如果活动已发布或进行中，直接删除会影响报名、签到、借用和处罚数据。限制只能删除 `cancelled` 活动，可以避免误删正常业务数据。删除时还使用事务按外键依赖顺序删除子表，保证数据一致性。

问题 8：为什么不用 `ON DELETE CASCADE`？

回答思路：

`ON DELETE CASCADE` 可以自动级联删除，但太隐式，误删风险高。课程项目希望展示事务删除过程，所以手动在存储过程中按顺序删除，逻辑更清楚，也方便答辩说明。

问题 9：视图有什么作用？

回答思路：

视图把复杂统计 SQL 封装起来。比如 `v_activity_summary` 聚合活动、社团、场地、报名、签到和借用数量；后端统计页面只需要查询视图，不需要重复写复杂 JOIN。这样统计口径统一，也体现数据库查询封装能力。

问题 10：如何处理借用逾期？

回答思路：

有两种处理。活动结算过程会把当前活动相关的过期 `borrowed` 借用单改为 `overdue`，并生成处罚。另一个是 MySQL Event `ev_mark_overdue_orders` 每 30 分钟自动扫描所有 `borrowed` 且超过应归还时间的借用单，改为 `overdue` 并插入处罚，同时用 `NOT EXISTS` 防止重复处罚。

问题 11：如何防止库存被并发扣成负数？

回答思路：

添加借用明细时，后端用 `SELECT ... FOR UPDATE` 锁住对应 `resource_item` 行，然后检查 `available_qty` 是否足够，再插入明细并扣减库存。这样并发请求会排队执行，避免两个事务同时看到同一份库存。

问题 12：二手交易如何防止一个商品被多人同时买？

回答思路：

创建订单时先 `SELECT product ... FOR UPDATE` 锁住商品行，确认状态还是 `on_sale`，再插入订单并把商品状态改为 `locked`。如果另一个人同时下单，必须等锁释放后再查，此时商品已不是 `on_sale`，就不能再下单。

问题 13：CHECK 约束有什么用？

回答思路：

CHECK 用于限制字段取值和业务边界，比如角色只能是三种，活动结束时间必须晚于开始时间，库存不能小于 0，信用分在 0 到 120 之间。它能在数据库层阻止非法数据进入。

问题 14：为什么有些状态字段用字符串而不是数字？

回答思路：

字符串状态可读性好，适合课程演示和调试。比如 `published`、`ongoing` 一眼能看懂。替代方案是数字编码或状态字典表，生产系统中如果状态很多可以考虑字典表。

问题 15：你这个设计满足第几范式？

回答思路：

大部分核心表满足第三范式。实体拆分清晰，非主属性依赖主键，比如学生资料在 `student`，社团资料在 `club`，活动报名关系在 `activity_registration`。也有少量为了查询效率保留的状态字段，如 `activity_registration.checkin_status`，这是性能和审计之间的折中。

问题 16：为什么 `activity_registration` 里有签到状态，`checkin` 表里也有状态？

回答思路：

报名表里的 `checkin_status` 是当前状态，方便活动详情和统计快速查询；`checkin` 表是操作记录，保存签到时间和操作人。一个偏当前状态，一个偏操作流水。

问题 17：审计日志为什么用 JSON？

回答思路：

不同业务对象变化字段不同，用 JSON 可以灵活保存 `old_data` 和 `new_data`，不需要为每类业务单独设计日志表。缺点是结构约束较弱，复杂查询不如普通列方便。

问题 18：为什么二手交易和社区是后加的 SQL 文件？

回答思路：

项目最初以活动和物资为核心，后来扩展成校园百事通，所以 `08_add_trade_community.sql` 是增量扩展脚本。这样既保留原系统，又能展示数据库扩展能力。

问题 19: `99_bootstrap_all.sql` 和 `99_bootstrap_workbench.sql` 有什么区别？

回答思路：

`99_bootstrap_all.sql` 使用 `SOURCE` 命令，适合 MySQL 命令行；`99_bootstrap_workbench.sql` 把所有 SQL 展开到一个文件，适合 Workbench 直接执行，因为 Workbench 对 `SOURCE` 不如命令行方便。

问题 20: 这个数据库设计最大的不足是什么？如何改进？

回答思路：

可以回答三个点：

1. 密码明文存储，应该改成哈希。
2. 场地时间冲突未检查，可以增加触发器或预约表。
3. 候补转正过程还未接入前端，可以增加取消报名并调用 `sp_promote_waitlist`。

这样回答比说“没有不足”更可信。

32. 答辩时推荐展示顺序

1. 先展示系统页面，让老师知道项目能跑。
2. 展示数据库 ER 关系或表结构。
3. 重点讲 `activity_registration` 触发器。
4. 演示学生报名：满员后自动进入候补。
5. 展示 `sp_finish_activity`，说明事务结算。
6. 演示统计视图页面。
7. 补充讲借用库存的 `FOR UPDATE` 并发控制。
8. 最后说扩展模块：二手交易和社区。

33. 可以背下来的总结

本项目数据库设计以统一用户体系为基础，把校园活动、物资借还、二手交易和社区互动统一到一个 MySQL 数据库中。核心业务规则不是只依赖前端，而是通过外键、唯一约束、CHECK、触发器、存储过程和事务共同保证。报名规则由触发器自动判断，活动结算和删除由存储过程保证原子性，统计查询通过视图封装，逾期借用通过事件定时处理。这样的设计既满足业务演示，也体现了数据库课程要求的约束设计、事务处理、数据库编程和统计查询能力。